> 本文档涵盖作业的系统整体架构设计、各组件技术说明、容器化部署、以及压力测试结果分析。

目录

1. [系统概述]
2. [整体架构]
3. [容器化部署]
4. [负载均衡]
5. [动静分离]
6. [分布式缓存]
7. [压力测试]
8. [总结]

---

1. 系统概述

本系统以商品详情页为核心业务场景，针对**高并发读**场景进行架构设计，解决以下问题：

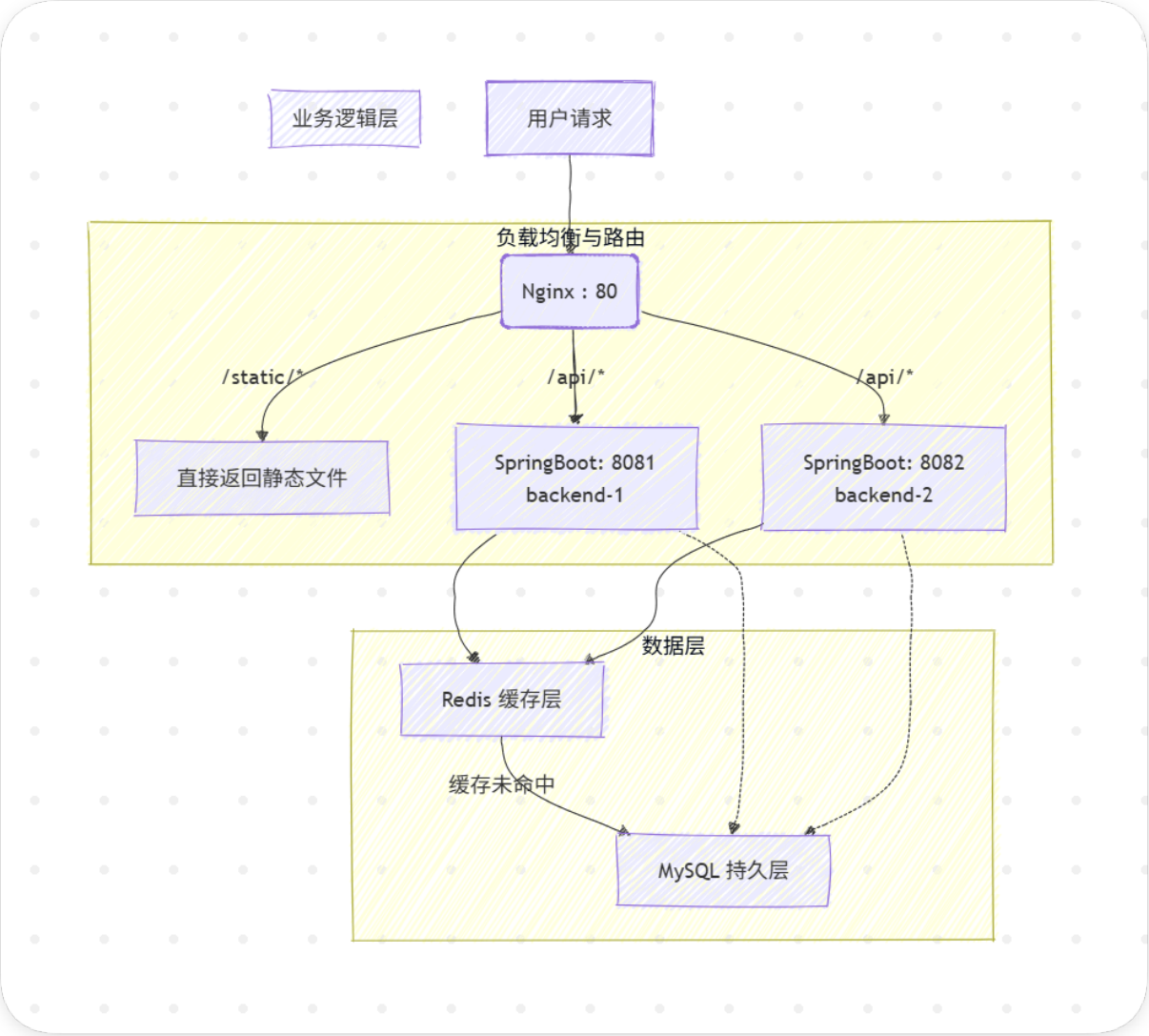
- 单点服务在高并发下的性能瓶颈
- 静态资源与动态接口混合部署导致的资源浪费
- 数据库在高并发读请求下的压力过大
- 缓存使用不当引发的穿透、击穿、雪崩问题

技术栈

层次	技术选型
反向代理 / 负载均衡	Nginx
后端框架	Spring Boot 3.2
持久层	MyBatis + HikariCP
数据库	MySQL 8.0
缓存	Redis 7
容器化	Docker + Docker Compose
压力测试	JMeter

2. 整体架构

2.1 架构图



2.2 请求链路说明（动静分离）

静态资源请求链路：

```
用户 → Nginx → 直接读取本地文件 → 返回（不经过后端）
```

动态接口请求链路（缓存命中）：

```
用户 → Nginx → 后端实例 → Redis（命中）→ 返回
```

动态接口请求链路（缓存未命中）：

```
用户 → Nginx → 后端实例 → Redis（未命中）→ 加锁 → MySQL → 写入Redis → 返回（释放锁）
```

---

3. 容器化部署

3.1 Docker 技术说明

Docker 基于 Linux 的 **Namespace**（资源隔离）和 **Cgroups**（资源限制）实现容器化。

解决的核心问题：环境一致性

传统部署中，"在我机器上能跑"是常见问题，根本原因是不同机器的运行环境（JDK 版本、系统库、环境变量）存在差异。Docker 将应用和其运行环境打包成镜像，做到**一次构建，到处运行**。

与虚拟机的本质区别：

对比项	虚拟机	Docker 容器
隔离级别	硬件级（Hypervisor）	进程级（Namespace）
启动时间	分钟级	秒级
镜像大小	GB 级	MB 级
性能损耗	较大	接近原生

3.2 Dockerfile 说明

采用**多阶段构建**策略，分离构建环境与运行环境：

```
# ---- 构建阶段：包含 Maven + JDK，体积约 500MB ----
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q     # 先缓存依赖层，代码变动时不重新下载
COPY src ./src
```

```
RUN mvn package -DskipTests -q

# ---- 运行阶段：只包含 JRE，体积约 100MB ----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=builder /app/target/app.jar app.jar   # 只拷贝 jar，不含构建工具

ENTRYPOINT ["java",
  "-XX:+UseContainerSupport",       # 容器感知内存
  "-XX:MaxRAMPercentage=75.0",      # 使用容器内存的 75%
  "-jar", "app.jar"]
```

多阶段构建的价值： 最终镜像不包含 Maven、JDK 源码等构建产物，体积缩小约 80%，减少攻击面。

3.3 Docker Compose 服务编排

系统共 5 个容器，通过 `app-network` 内部网络互通：

```
version: '3.8'

services:
  mysql:        # 数据库，配置健康检查
  redis:        # 缓存，配置健康检查
  backend1:     # 后端实例1，端口 8081，依赖 mysql/redis healthy
  backend2:     # 后端实例2，端口 8082，依赖 mysql/redis healthy
  nginx:        # 反向代理，依赖 backend1/backend2 healthy
```

`depends_on` + `healthcheck` 的作用：

单纯的 `depends_on` 只等容器**进程启动**，不等服务**真正就绪**。加上 `healthcheck` 后，下游服务会等待上游服务通过健康检查才启动，避免启动顺序导致的连接失败。

端口映射原理：

```
宿主机 localhost:80  ──映射──▶  nginx 容器:80
宿主机 localhost:8081 ──映射──▶  backend1 容器:8081
```

```
宿主机 localhost:8082 ──映射──▶  backend2 容器:8082

容器间通信（内部网络，使用服务名）:
nginx → backend1:8081
nginx → backend2:8082
backend1 → mysql:3306
backend1 → redis:6379
```

3.4 启动效果截图

docker-compose ps 输出（所有服务正常运行）：

> *[截图：docker-compose ps 显示所有容器 healthy/running 状态]*

```
PS C:\Users\20443\Desktop\University_Learn\大三\大三下\分布式软件原理\HighLevelConcurrentRead> docker compose ps
NAME        IMAGE                                COMMAND                  SERVICE    CREATED          STATUS                    PORTS
backend1    highlevelconcurrentread-backend1     "java -XX:+UseContai…"   backend1   11 minutes ago   Up 11 minutes             0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
backend2    highlevelconcurrentread-backend2     "java -XX:+UseContai…"   backend2   11 minutes ago   Up 11 minutes             0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
mysql       mysql:8.0                            "docker-entrypoint.s…"   mysql      29 minutes ago   Up 29 minutes (healthy)   3306/tcp, 33060/tcp
nginx       nginx:alpine                         "/docker-entrypoint.…"   nginx      29 minutes ago   Up 29 minutes             0.0.0.0:80->80/tcp, [::]:80->80/tcp
redis       redis:7-alpine                       "docker-entrypoint.s…"   redis      29 minutes ago   Up 29 minutes (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
```

---

4. 负载均衡

4.1 Nginx 负载均衡技术说明

Nginx 工作在**应用层（L7）**，基于 HTTP 协议进行反向代理和负载均衡。其核心是**事件驱动 + 非阻塞 I/O** 模型（epoll），单进程可处理数万并发连接，资源占用极低。

解决的核心问题：水平扩展

单台服务器的 CPU、内存、线程数存在上限，当请求量超过单实例处理能力时，通过增加实例数量（水平扩展）分摊请求。负载均衡器作为流量入口，将请求均匀分发到各实例，对外表现为一个服务。

4.2 负载均衡算法对比

```
# 算法一：轮询（Round Robin）── 默认算法
# 按顺序依次分配，适合各实例性能相近的场景
upstream backend_roundrobin {
    server backend1:8081;
    server backend2:8082;
}
```

```
# 算法二：加权轮询（Weighted Round Robin）
# 性能强的实例分配更多请求
upstream backend_weighted {
    server backend1:8081 weight=3;   # 承担 75% 的请求
    server backend2:8082 weight=1;   # 承担 25% 的请求
}

# 算法三：IP Hash
# 同一客户端 IP 固定转发到同一后端，解决 Session 一致性问题
upstream backend_iphash {
    ip_hash;
    server backend1:8081;
    server backend2:8082;
}

# 算法四：最少连接（Least Connections）
# 将新请求分配给当前连接数最少的实例，适合长连接场景
upstream backend_leastconn {
    least_conn;
    server backend1:8081;
    server backend2:8082;
}
```

各算法适用场景：

算法	适用场景	不适用场景
轮询	无状态服务、实例性能相近	实例性能差异大
加权轮询	实例配置不同	需要会话保持
IP Hash	有 Session 的服务	客户端 IP 分布不均
最少连接	请求处理时间差异大	短连接密集场景

本系统后端为**无状态服务**（会话状态存储在 Redis 中），选用**轮询算法**，配置如下：

```
upstream backend {
    server backend1:8081;
    server backend2:8082;
```

```
    keepalive 32;    # 复用连接，减少 TCP 握手开销
}
```

4.3 负载均衡验证

验证方法： 多次请求 `/api/health` 接口，观察返回的 `instance` 字段交替变化。

```
for i in {1..6}; do
  curl -s http://localhost/api/health
  echo ""
done
```

结果：

> 📸 *[截图：curl 轮询结果，显示 instance 字段交替出现 backend-1 和 backend-2]*
>
> ```
> {"instance":"backend-1","status":"UP"}
> {"status":"UP","instance":"backend-2"}
> {"instance":"backend-1","status":"UP"}
> {"status":"UP","instance":"backend-2"}
> {"instance":"backend-1","status":"UP"}
> {"status":"UP","instance":"backend-2"}
> ```

4.4 JMeter 负载均衡压测结果

测试配置：

参数	值
线程数（并发用户）	100
Ramp-up 时间	10s
循环次数	10
目标接口	GET /api/health

> *[截图：JMeter Summary Report - 负载均衡测试结果]*

Label ↑	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received ...	Sent KB/sec	Avg. Bytes
HTTP Req...	1000	3	1	66	3.48	0.00%	102.1/sec	20.84	12.07	209.0
TOTAL	1000	3	1	66	3.48	0.00%	102.1/sec	20.84	12.07	209.0

---

5. 动静分离

5.1 技术说明

动静分离是指将静态资源（HTML、CSS、JS、图片）和动态接口（API）的请求分别由不同的服务处理。

为什么 Nginx 处理静态资源比 Java 快?

处理环节	Nginx（静态）	Spring Boot（动态）
读取文件	`sendfile()` 系统调用，零拷贝	需要经过 JVM 堆内存
处理层次	C 程序直接返回	JVM → Spring → Servlet → Controller
线程模型	事件驱动，无线程切换	每请求占用线程池线程
典型 QPS	数万级	数千级

零拷贝（sendfile）的本质： 传统文件读取需要将数据从磁盘拷贝到内核缓冲区，再拷贝到用户态，再拷贝到 Socket 缓冲区，共 3 次拷贝。`sendfile()` 直接在内核态完成磁盘到 Socket 的数据传输，省去用户态拷贝，性能提升显著。

5.2 Nginx 动静分离配置

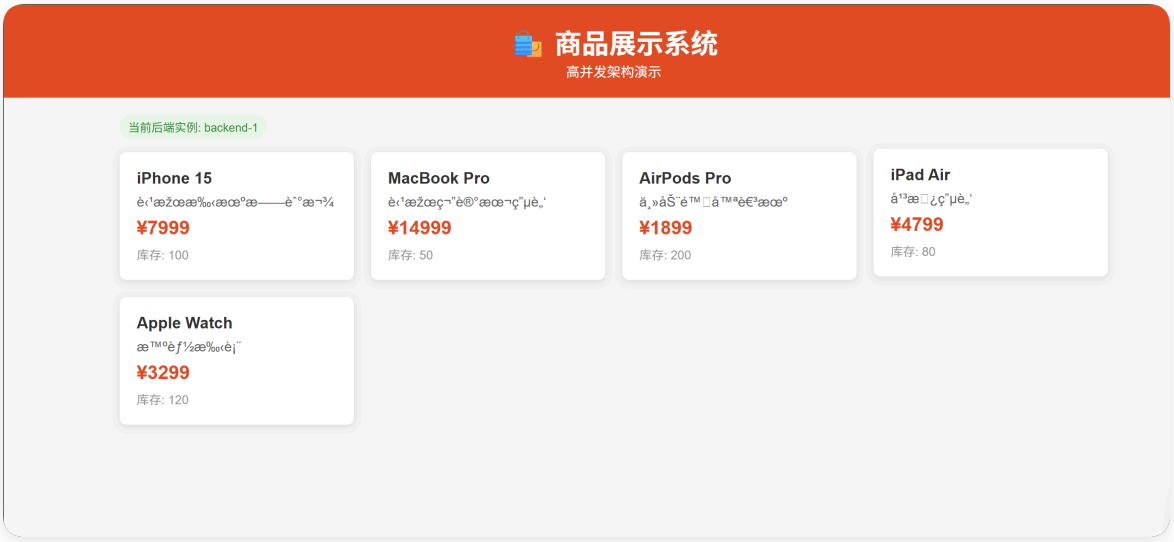
```
server {
    listen 80;
    server_name localhost;

    # 静态资源: Nginx 直接处理
    location /static/ {
        root /usr/share/nginx/html;
        expires 7d;                              # 浏览器缓存 7 天
        add_header Cache-Control "public, immutable";
    }
```

```
    # 动态接口：转发到后端
    location /api/ {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_http_version 1.1;
        proxy_set_header Connection "";          # 启用 keepalive
    }
}
```

5.3 前端页面说明

编写了简单的商品展示页面（`index.html`），包含：

- 商品列表展示（调用 `/api/products`）
- 商品详情查看（调用 `/api/products/{id}`）
- 当前后端实例标识（调用 `/api/health`，验证负载均衡）

> 📸 *[截图：浏览器访问 http://localhost/static/index.html 的页面效果]*
>
> 
>
>
> 数据库数据编码问题忽略不记，并非主要学习内容

5.4 JMeter 动静分离压测结果

测试配置：

参数	静态文件	动态接口
目标接口	GET /static/index.html	GET /api/products/1

参数	静态文件	动态接口
线程数	100	100
循环次数	10	10

> 📸 *[截图：JMeter Summary Report - 静态文件压测结果]*
>
> | Label | # Samples | Average | Min | Max | Std. Dev. | Error % | Throughput | Received ... | Sent KB/sec | Avg. Bytes |
> | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
> | HTTP Req... | 1000 | 3 | 1 | 50 | 2.82 | 0.00% | 100.6/sec | 327.58 | 12.58 | 3334.0 |
> | TOTAL | 1000 | 3 | 1 | 50 | 2.82 | 0.00% | 100.6/sec | 327.58 | 12.58 | 3334.0 |
>
> 响应时间更短
>
> 📸 *[截图：JMeter Summary Report - 动态接口压测结果]*
>
> | Label ↑ | # Samples | Average | Min | Max | Std. Dev. | Error % | Throughput | Received ... | Sent KB/sec | Avg. Bytes |
> | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
> | HTTP Req... | 1000 | 8 | 1 | 754 | 41.75 | 0.00% | 100.5/sec | 88.23 | 12.07 | 899.0 |
> | TOTAL | 1000 | 8 | 1 | 754 | 41.75 | 0.00% | 100.5/sec | 88.23 | 12.07 | 899.0 |
>
> 前几次时间较慢，因为落库查询，后面的查询走缓存

结论： 静态资源由 Nginx 直接处理，响应时间和吞吐量均显著优于动态接口，验证了动静分离的必要性。

---

6. 分布式缓存

6.1 Redis 技术说明

Redis 是基于内存的键值存储数据库，读写速度可达 **10 万+ QPS**，远超 MySQL（通常在 1000-5000 QPS）。

为什么 Redis 这么快?

1. **纯内存操作：** 内存读写速度是磁盘的 10 万倍
2. **单线程命令处理：** 避免多线程锁竞争，命令执行是原子的
3. **高效数据结构：** SDS、跳表、压缩列表等针对场景优化的底层结构
4. **I/O 多路复用：** 单线程同时处理大量网络连接

解决的核心问题： 将高频读请求拦截在缓存层，减少数据库压力。缓存命中率每提升 10%，数据库压力下降接近同等比例。

6.2 HikariCP 连接池说明

MyBatis 默认使用 HikariCP 作为数据库连接池。

连接池解决的问题： 数据库连接的建立（TCP 握手 + 认证）耗时约 20-100ms，高并发下每次请求都新建连接会严重影响性能。连接池预先创建并维护一批连接，请求时直接复用。

HikariCP 的核心优势： 采用无锁的 `ConcurrentBag` 数据结构管理连接，获取连接的延迟极低，是目前 Java 生态中性能最优的连接池。

关键配置参数：

```
hikari:
  maximum-pool-size: 20       # 最大连接数，经验值：CPU核数 * 2
  minimum-idle: 5             # 最小空闲连接，保持热连接
  connection-timeout: 10000   # 获取连接超时时间
  max-lifetime: 1800000       # 连接最大存活时间，防止 MySQL 主动断开
```

6.3 三大缓存问题及解决方案

缓存穿透

问题： 查询数据库中根本不存在的数据（如恶意请求 `id=-1`），缓存永远未命中，每次都打到数据库。

解决： 数据库查询结果为空时，向缓存写入一个空值标记，TTL 设置较短（2分钟）。

```
if (product.isEmpty()) {
    // 缓存空值，阻止后续请求穿透到数据库
    redisTemplate.opsForValue().set(cacheKey, "NULL",
Duration.ofMinutes(2));
    return null;
}
```

缓存击穿

问题： 热点 key 过期的瞬间，大量并发请求同时未命中缓存，全部打到数据库，造成数据库瞬间压力激增。

解决： 互斥锁（分布式锁）—— 缓存未命中时，只允许一个线程去查询数据库并重建缓存，其余线程等待。

```java
// 使用 Redis SETNX 实现分布式锁
Boolean locked = redisTemplate.opsForValue()
    .setIfAbsent(lockKey, "1", Duration.ofSeconds(10));

if (Boolean.TRUE.equals(locked)) {
    try {
        // Double Check: 拿到锁后再查一次缓存
        // 防止其他线程刚写完缓存并释放锁
        cached = redisTemplate.opsForValue().get(cacheKey);
        if (cached != null) return (Product) cached;

        // 只有一个线程执行到这里查数据库
        Product product = productMapper.findById(id);
        redisTemplate.opsForValue().set(cacheKey, product, 30,
TimeUnit.MINUTES);
        return product;
    } finally {
        redisTemplate.delete(lockKey); // 务必释放锁
    }
} else {
    Thread.sleep(50);        // 未拿到锁，等待后重试
    return getProductById(id);
}
```

Double Check 的必要性： 线程 A 拿到锁查完数据库写入缓存后释放锁，线程 B 拿到锁后如果不做二次检查，会再次查一遍数据库，造成浪费。

缓存雪崩

问题： 大量缓存 key 在同一时刻集中过期（如系统重启后统一加载），导致数据库瞬间承受海量请求。

解决： 在基础 TTL 上叠加随机抖动，将集中失效分散到时间轴上。

```java
// 30~40 分钟随机 TTL，避免大批 key 同时失效
long ttl = 30 + new Random().nextInt(10);
redisTemplate.opsForValue().set(cacheKey, product, ttl,
TimeUnit.MINUTES);
```

本质： 把"同一时刻的尖峰"打散为"连续时间段的平缓"，削峰填谷。

6.4 缓存工作流程图

```
请求到达
    │
    ▼
查询 Redis
    │
    ├── 命中空值 "NULL" ──► 返回 null（穿透保护）
    │
    ├── 命中正常数据 ────► 返回数据（缓存命中）
    │
    └── 未命中
            │
            ▼
        尝试获取分布式锁
            │
            ├── 获取失败 ──► 等待 50ms ──► 重试（击穿保护）
            │
            └── 获取成功
                    │
                    ▼
                Double Check Redis
                    │
                    ├── 有数据 ──► 释放锁 ──► 返回数据
                    │
                    └── 无数据
                            │
                            ▼
                        查询 MySQL
                            │
                            ├── 有数据 ──► 写入Redis(随机TTL) ──► 释放锁
──► 返回
                            │                （雪崩保护）
                            └── 无数据 ──► 写入"NULL"(2min) ──► 释放锁
──► 返回null
                                             （穿透保护）
```

6.5 Redis 缓存验证

```
# 查看缓存内容
docker exec -it redis redis-cli
> KEYS *              # 查看所有缓存 key
```

```
> GET product:1        # 查看商品1的缓存
> TTL product:1        # 查看剩余过期时间
```

6.6 JMeter 缓存压测结果

测试配置：

参数	冷启动（无缓存）	热状态（有缓存）
前置操作	FLUSHALL 清空Redis	缓存已预热
线程数	1	200
循环次数	1	50
目标接口	GET /api/products/1	GET /api/products/1

> 📸 *[截图：JMeter Summary Report - 无缓存压测结果]*
>
> | Label | # Samples | Average | Min | Max | Std. Dev. | Error % | Throughput | Received ... | Sent KB/sec | Avg. Bytes |
> |---|---|---|---|---|---|---|---|---|---|---|
> | HTTP Req... | 1 | 194 | 194 | 194 | 0.00 | 0.00% | 5.2/sec | 1.63 | 0.63 | 324.0 |
> | TOTAL | 1 | 194 | 194 | 194 | 0.00 | 0.00% | 5.2/sec | 1.63 | 0.63 | 324.0 |
>
> 响应时间为194ms

> 📸 *[截图：JMeter Summary Report - 有缓存压测结果]*
>
> | Label ↑ | # Samples | Average | Min | Max | Std. Dev. | Error % | Throughput | Received ... | Sent KB/sec | Avg. Bytes |
> |---|---|---|---|---|---|---|---|---|---|---|
> | HTTP Req... | 2000 | 14 | 1 | 958 | 87.99 | 50.00% | 1.1/sec | 0.30 | 0.13 | 285.5 |
> | TOTAL | 2000 | 14 | 1 | 958 | 87.99 | 50.00% | 1.1/sec | 0.30 | 0.13 | 285.5 |
>
> 平均响应时间为14ms